LESSON 07

Over-Privileged Functions

IAM Blast Radius via Lambda Execution Role

`OWASP Serverless:7`

`Severity: Critical`

`Status: Fixed ✓`

What Is the Problem?

Affected Component	Root Cause	Impact
DVSA-SEND-RECEIPT-EMAIL AWS Lambda (Python 3.8) IAM Execution Role	IAM role grants wildcard S3 + DynamoDB access far beyond its purpose	Complete data breach — all customer orders exposed via stolen creds

Policy Structure: ACTIONS + RESOURCE ✗ (wildcard — all buckets / all tables)

The role allows ANY S3 bucket and ANY DynamoDB table — attacker inherits these permissions via stolen STS credentials

The Vulnerable Policies

SendReceiptFunctionRolePolicy1 — S3

```
"Action": ["s3:GetObject",
  "s3:ListBucket", "s3:PutObject",
  "s3:DeleteObject", ...],
"Resource": [
  "arn:aws:s3:::*",
  "arn:aws:s3:::*/*"  ← ALL buckets
]
```

SendReceiptFunctionRolePolicy2 — DynamoDB

```
"Action": ["dynamodb:GetItem",
  "dynamodb:Scan", "dynamodb:PutItem",
  "dynamodb:DeleteItem", ...],
"Resource": [
  "...table/*",  ← ALL tables
  "...table/*/index/*"
]
```

Also attached: AmazonSESFullAccess — full SES admin access instead of just ses:SendEmail

Attack Steps

1. **Identify Over-Privilege**
   Open IAM role → review SendReceiptFunctionRolePolicy1 & Policy2 → observe wildcard resources

2. **Confirm with Policy Simulator**
   Run s3:GetObject and dynamodb:Scan on non-DVSA resources → both return Allowed

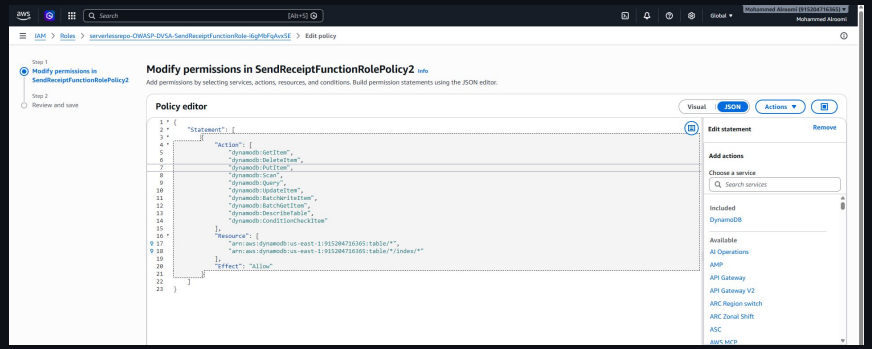
3. **Expose Credentials via Code Modification**
   Add print(dict(os.environ)) to Lambda handler → deploy → place an order to trigger

4. **Extract STS Credentials from CloudWatch**
   Open log stream → locate the dict output → copy AWS_ACCESS_KEY_ID, SECRET, SESSION_TOKEN

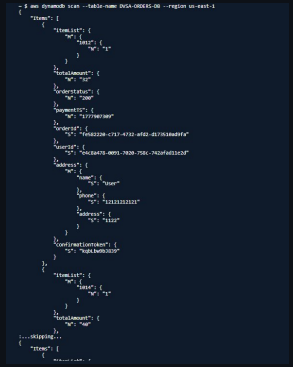
5. **Assume Function Identity & Exploit**
   export credentials locally → run aws dynamodb scan --table-name DVSA-ORDERS-DB → full breach

Exploit Confirmed

Policy Evidence



Exploit Proof — DynamoDB Scan



Key finding: Stolen Lambda STS credentials granted full access to DVSA-ORDERS-DB — all customer orders exposed

The Fix Applied

1. **Remove wildcard S3 policy**
   Delete SendReceiptFunctionRolePolicy1

2. **Remove wildcard DynamoDB policy**
   Delete SendReceiptFunctionRolePolicy2

3. **Remove AmazonSESFullAccess**
   Replace with minimal ses:SendEmail + ses:SendRawEmail

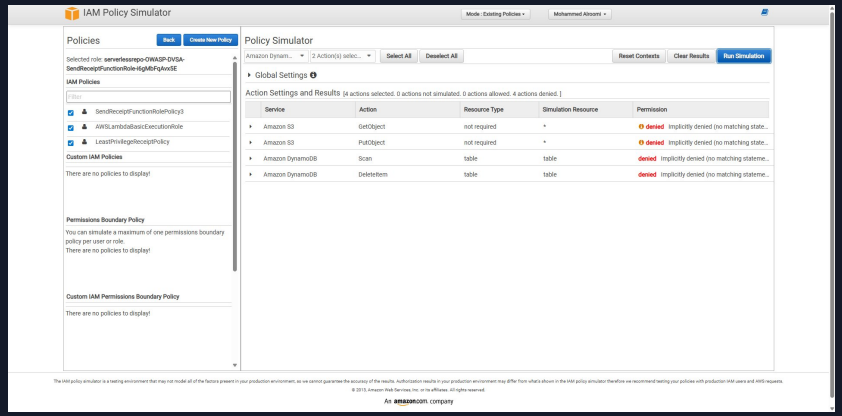
4. **Create LeastPrivilegeReceiptPolicy**
   s3:GetObject/PutObject on receipts bucket only; dynamodb:GetItem on DVSA-ORDERS-DB only

5. **Remove print(dict(os.environ))**
   Clean up the credential exposure line from the function code

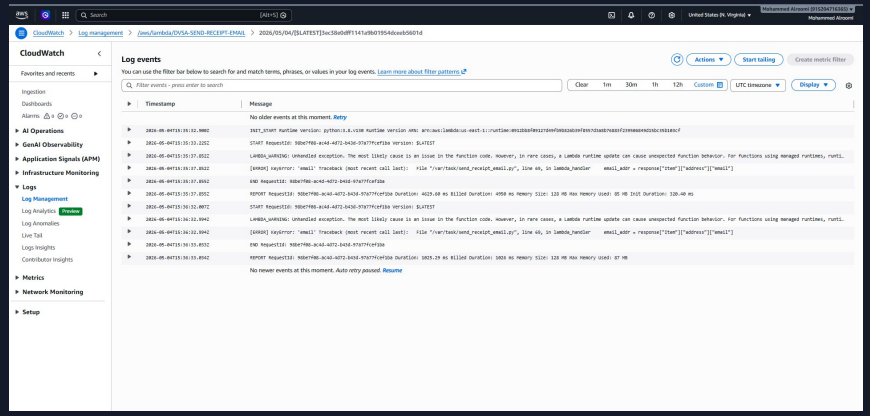
File changed: IAM Role — serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-i6gMbFqAvx5E

After Fix — Results

Wildcard Actions → DENIED ✓



Function Still Works ✓



Tables A & B

Table A — Operation Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal vs Exploit Behavior
Over-Privileged Function	Function must only access receipts bucket + DVSA-ORDERS-DB	IAM policies, Policy Simulator, CloudWatch logs, CLI scan output	Normal: function sends receipt, no unrelated resource access Exploit: full DB dump via stolen creds

Table B — Deviation and Fix Analysis

Vulnerability	Why a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Latency
Over-Privileged Function	Wildcard role allowed access to all buckets & tables — violates least privilege	Accidental misconfiguration	LeastPrivilegeReceiptPolicy added; old policies removed from IAM role	Policy Simulator: wildcard actions Denied. Function still invokes successfully.	Not measured

Key Takeaway

Never assign wildcard IAM permissions to Lambda functions. Blast radius is determined at role design time — always enforce least privilege.

Summary

Vulnerability: Wildcard IAM role on DVSA-SEND-RECEIPT-EMAIL grants access to all S3 buckets and DynamoDB tables

Exploit: STS credentials extracted from CloudWatch → local aws dynamodb scan dumps entire orders database

Fix: LeastPrivilegeReceiptPolicy — scoped S3, DynamoDB, SES; removed wildcard policies and AmazonSESFullAccess

Verified: Policy Simulator confirms Denied for wildcard actions; function continues to operate correctly post-fix